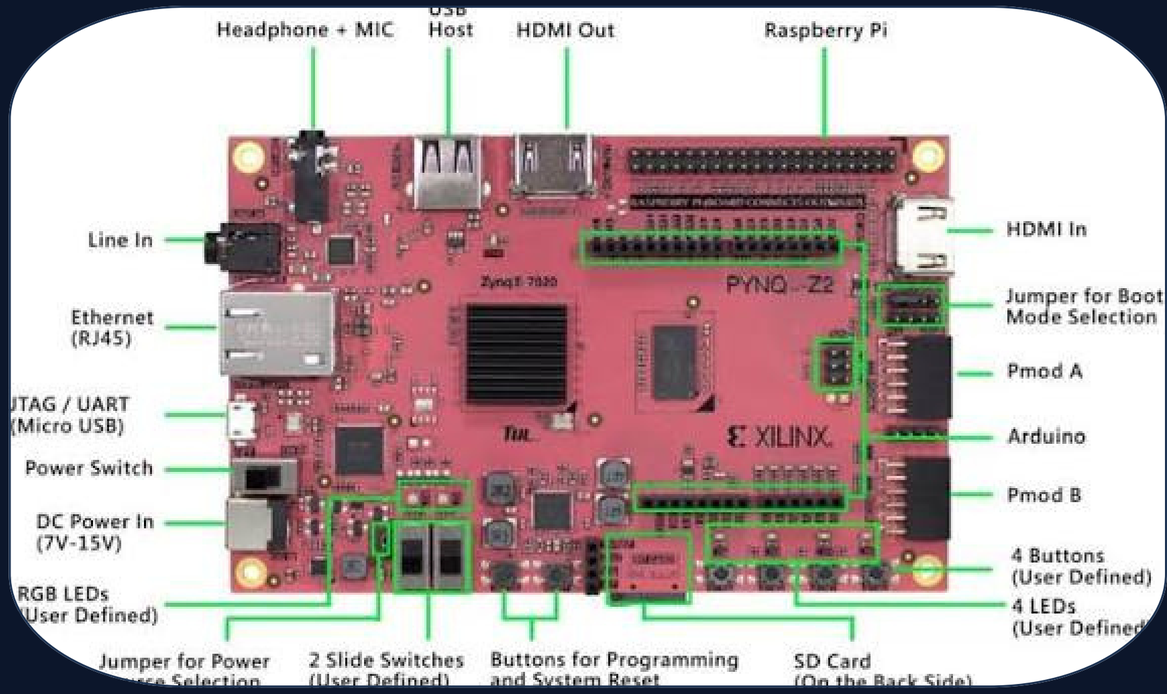
FPGA • EMBEDDED SYSTEMS • COMPUTER VISION

Real-Time Object Detection Using YOLOv3-Tiny on PYNQ-Z2

Zynq-7000 | PYNQ-Z2 | AXI DMA | YOLOv3-Tiny

A hardware/software co-design demonstration combining a Zynq processing system, AXI-based data movement and deep-learning-based object detection.



Designed and implemented by Kundan Singh

Project presentation

Project overview

A Zynq-based computer-vision demonstration with an AXI DMA streaming subsystem.

Platform

PYNQ-Z2 development board
Zynq-7000 SoC / ARM Cortex-A9

Hardware architecture

AXI DMA • AXI4-Stream FIFO
AXI Interconnect • PS reset

Vision pipeline

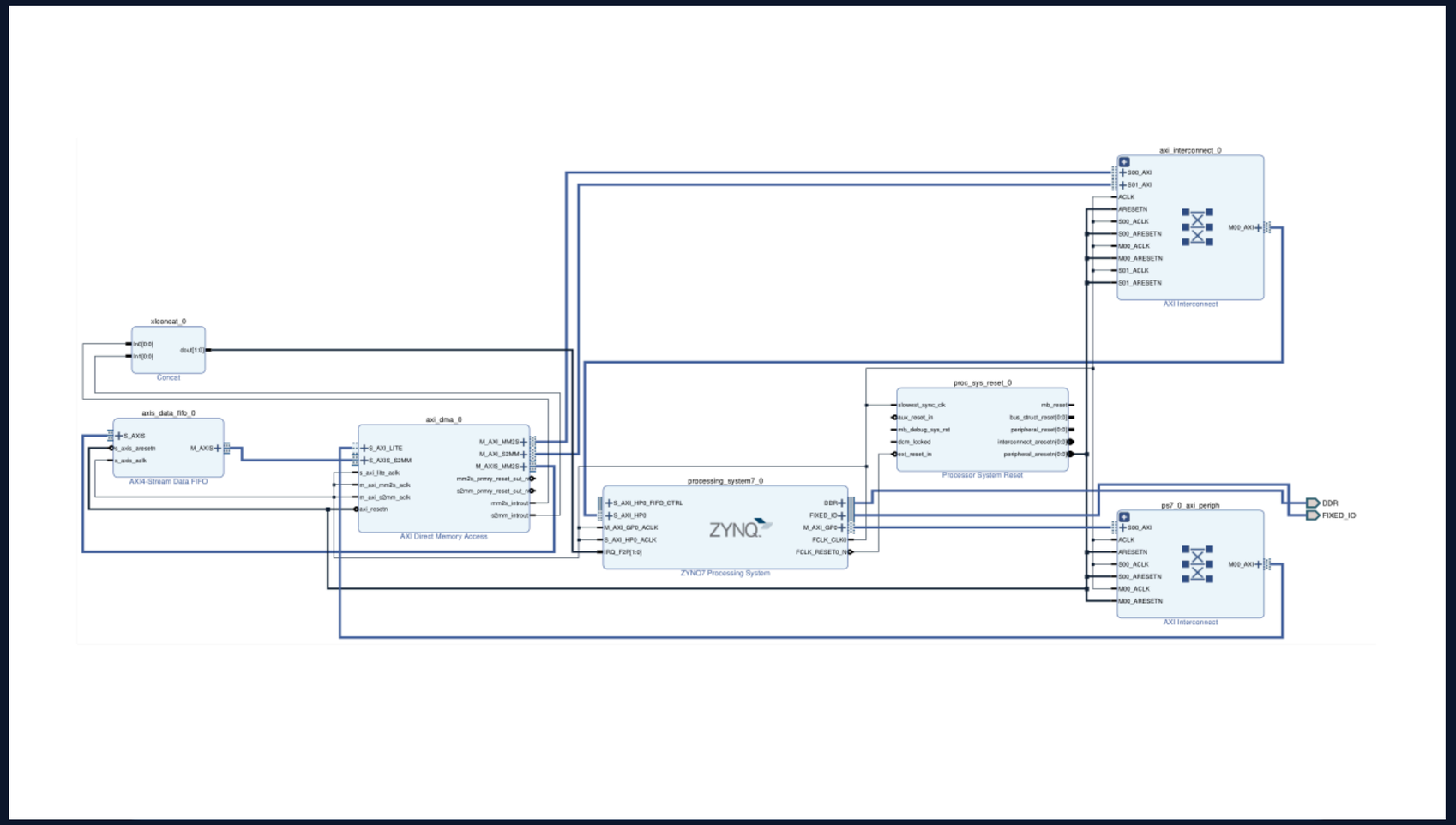
USB camera • OpenCV
YOLOv3-Tiny • Python

Project goals

- Integrate the Zynq Processing System with programmable-logic interfaces using AXI infrastructure.
- Demonstrate memory-to-stream and stream-to-memory transfers using AXI DMA.
- Run live object detection on camera frames and display class labels, bounding boxes and confidence scores.

Custom SoC block design

Vivado IP Integrator architecture supplied for this project (source: uploaded SoC diagram, page 1).



Processing System

processing_system7_0
Zynq-7000 PS

DMA + stream

axi_dma_0
axis_data_fifo_0

Infrastructure

AXI interconnects
proc_sys_reset_0
xlconcat_0

Zynq-7000: processing system and programmable logic

The PS executes software; the PL hosts the hardware interfaces and streaming infrastructure.

Processing System (PS)

- ARM Cortex-A9 processing subsystem
- DDR memory access
- M_AXI_GP0 control interface
- S_AXI_HP0 high-performance memory interface

AXI interfaces

Programmable Logic (PL)

- AXI DMA and AXI4-Stream interfaces
- AXI4-Stream Data FIFO
- AXI interconnect fabric
- Clock, reset and interrupt connectivity

Design interpretation: the Zynq PS provides the processor, DDR access, clocks and PS-side AXI ports. The PL-side IP blocks implement DMA control/data paths and streaming connectivity. The uploaded diagram does not show a dedicated YOLO inference accelerator in the PL.

AXI DMA: two transfer directions

The AXI DMA bridges memory-mapped DDR transactions and AXI4-Stream data.

MM2S — Memory-Mapped to Stream

Reads a source buffer from DDR and emits an AXI4-Stream data stream toward the PL-side interface.

S2MM — Stream to Memory-Mapped

Receives AXI4-Stream data from the PL-side interface and writes it into a destination buffer in DDR.



In the Python notebook, sendchannel maps to MM2S and recvchannel maps to S2MM. The transfer calls request DMA movement; the hardware stream path must be connected and configured to produce/consume the expected data.

Supporting IP: FIFO, interconnect, reset and interrupts

These blocks provide the infrastructure needed for coordinated AXI operation.

AXI4-Stream Data FIFO

Buffers stream beats and uses AXI4-Stream handshaking to manage data flow.

AXI Interconnects

Route AXI transactions between PS interfaces, DMA control and memory ports.

Processor System Reset

Distributes synchronized reset signals to connected AXI peripherals and interfaces.

XLConcat

Combines interrupt signals into a wider interrupt bus.

Clock and reset signals visible in the design

- FCLK_CLK0: PS-generated fabric clock used by PL-side logic and AXI clock domains as configured.
- FCLK_RESET0_N: PS reset output feeding the reset infrastructure; proc_sys_reset_0 generates synchronized reset outputs.

Software and computer-vision pipeline

Python, OpenCV and YOLOv3-Tiny run on the ARM processing system in the current implementation.

1 Camera capture

USB camera
640 × 480 frames

2 Enhancement

LAB colour space
CLAHE on L channel

3 DMA demo path

64 × 64 resize
Grayscale • 4,096 bytes

4 YOLO inference

416 × 416 input
OpenCV DNN on CPU

5 Post-processing

Confidence threshold
NMS • bounding boxes

Output

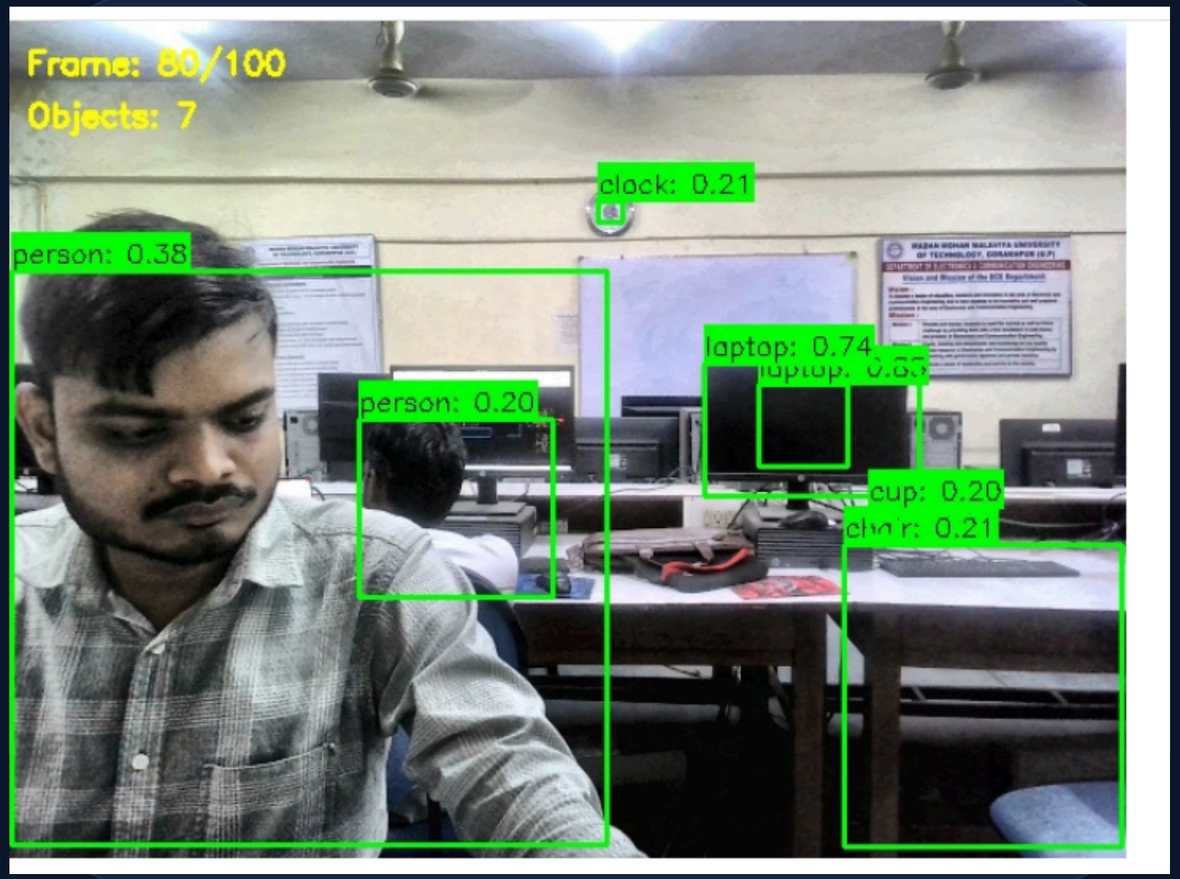
Annotated frame with class name, confidence score and bounding box.

Logging

Frame counter, object count and saved image: latest_detection.jpg.

Live detection results

Actual laboratory output screenshots from the project demonstration.

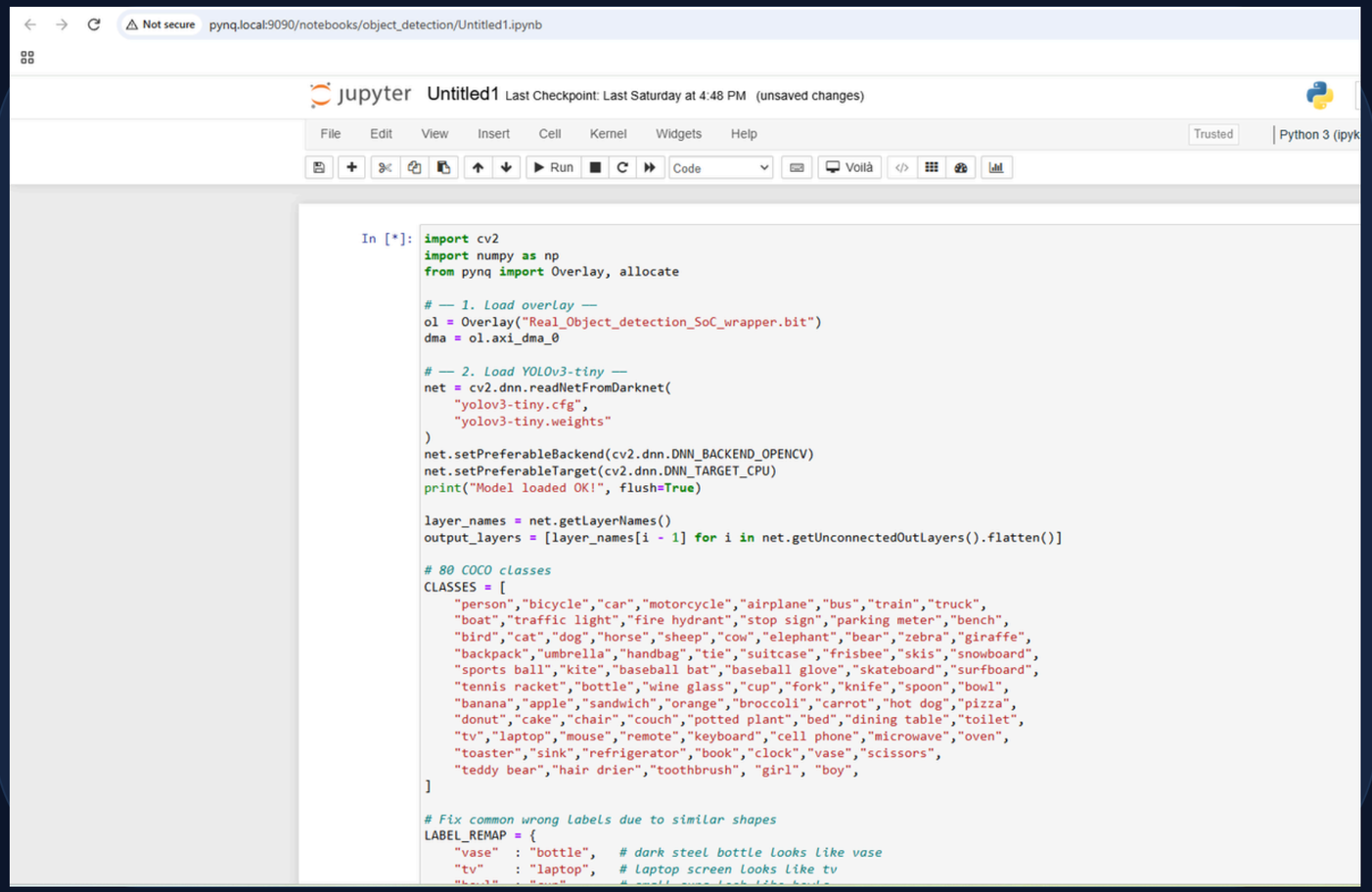




Observed labels include person, laptop, chair, cup and clock. Confidence scores vary by frame and scene conditions.

Implementation on PYNQ-Z2

Notebook and deployment files used for the demonstration.



FPGA overlay

Real_Object_detection_SoC_wrapper.bit
Matching .hwh metadata

YOLO model files

yolov3-tiny.cfg
yolov3-tiny.weights

Runtime

PYNQ Python • OpenCV DNN
USB camera • Jupyter Notebook

Current implementation: what runs where

Clear hardware/software partitioning is important when describing the project.

Currently on ARM CPU

- Python control and camera capture
- CLAHE image enhancement
- YOLOv3-Tiny inference via OpenCV DNN
- Detection post-processing and drawing

FPGA / PL infrastructure in the design

- AXI DMA MM2S and S2MM channels
- AXI4-Stream FIFO
- AXI interconnect and reset logic
- PS–PL clock, memory and interrupt connectivity

Important scope note: the current code sets DNN_TARGET_CPU. The DMA routine transfers a 64 × 64 grayscale buffer, but returns the enhanced original frame for YOLO inference. The uploaded block diagram does not show a dedicated YOLO accelerator IP in PL. Therefore, describe this as a SoC-integrated object-detection demonstration with AXI DMA data-transfer infrastructure—not FPGA-accelerated YOLO inference.

Next steps and possible hardware acceleration

A roadmap for extending the current SoC into a more complete FPGA computer-vision pipeline.

1. Validate DMA stream

Verify MM2S/S2MM connectivity, TLAST, TKEEP, TREADY and transfer lengths.

2. Add PL preprocessing

Implement grayscale conversion, resizing or image normalization in custom RTL/HLS IP.

3. Measure performance

Compare CPU-only and hardware-assisted latency, throughput and resource use.

Potential extension: build an AXI4-Stream accelerator between DMA MM2S and S2MM, then benchmark the complete PS–PL pipeline using repeatable input frames.

PROJECT SUMMARY

REAL-TIME OBJECT DETECTION SoC

Zynq-based hardware/software integration for live computer vision.

- Zynq-7000 / PYNQ-Z2 platform
- AXI DMA + AXI4-Stream FIFO infrastructure
- Python, OpenCV and YOLOv3-Tiny object detection
- Live laboratory demonstration with annotated output

Designed and implemented by

Kundan Singh

GitHub project presentation

Thank you